

Subset Sum and DP

Subset Sum

- ◆ We are given a set of integers — $A = \{a_1, a_2, \dots, a_n\}$.
- ◆ Is there a subset of sum Zero.

Subset Sum

- ◆ We are given a set of integers — $A = \{a_1, a_2, \dots, a_n\}$.
- ◆ Is there a **subset of sum Zero**.

Example — 2, -5, 1, 8, -4, -6

Subset Sum

- ◆ We are given a set of integers — $A = \{a_1, a_2, \dots, a_n\}$.
- ◆ Is there a **subset of sum Zero**.

Example — 2, -5, 1, 8, -4, -6



Subset Sum

- ◆ We are given a set of integers — $A = \{a_1, a_2, \dots, a_n\}$.
- ◆ Is there a **subset of sum Zero**.

Example — 2, -5, 1, 8, -4, -6

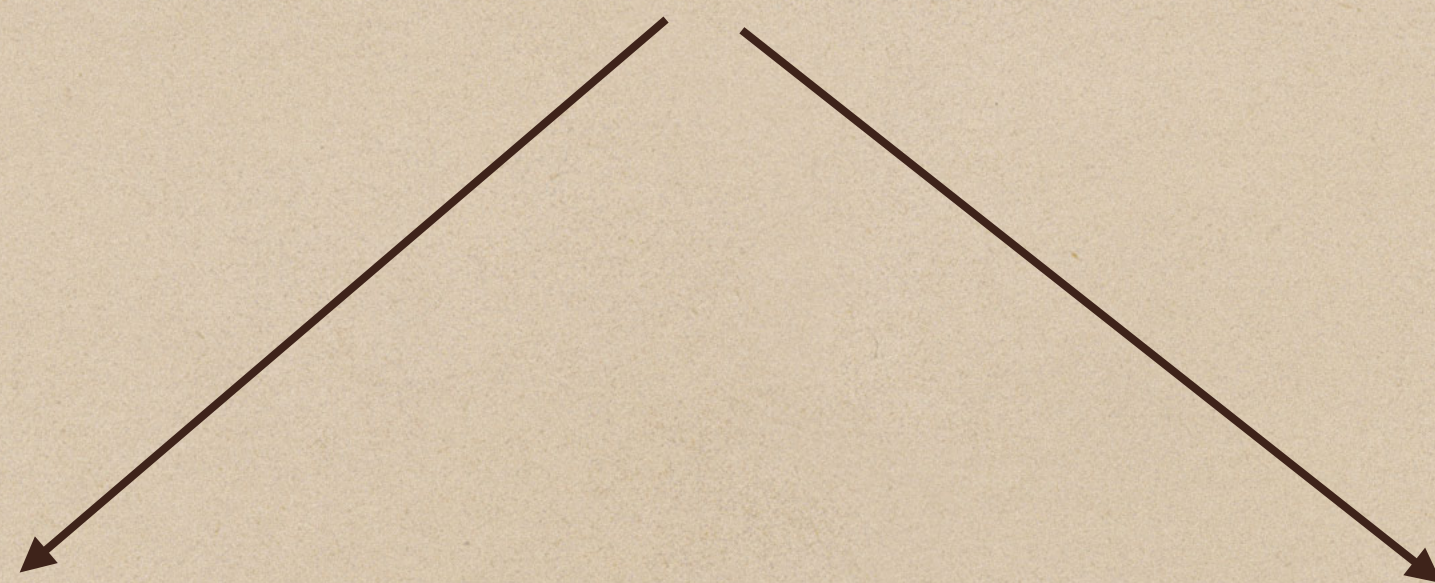


$\langle 2, 7, -4, -6 \rangle$

Guessing part

Solutions — (a) containing a_n ; (b) not containing a_n .

2, -5, 1, 8, -4, -6



Is there a subset of sum 0?

Is there a subset of sum 6?

Subproblems

- ◆ Subproblem — $A_j = \{a_1, \dots, a_n\}$
- ◆ Let's set the number to be N .
 - ◆ Is there a subset of A_j , with sum N ?
 - ◆ # of distinct subproblems = $n \times \sum_i a_i$

Subproblems

◆ Subproblem — $A_j = \{a_1, \dots, a_n\}$

◆ Let's set the number to be N .

◆ Is there a subset of A_j , with sum N ?

◆ # of distinct subproblems $= n \times \sum_i a_i$

ALG (j, N)

→ ALG ($j - 1, N$)

Or

ALG ($j - 1, n - a_j$)

Subproblems

- ◆ Subproblem — $A_j = \{a_1, \dots, a_n\}$

ALG (j, N)

- ◆ Let's set the number to be N .

→ ALG ($j - 1, N$)

Or

- ◆ Is there a subset of A_j , with sum N ?

ALG ($j - 1, n - a_j$)

- ◆ # of distinct subproblems = $n \times \sum_i a_i$

— Pseudo-polynomial time algorithm... ?

— Polynomial time alg. supposed to take time $\text{poly}(n, \text{no. of bits of } a_1, a_2, \dots)$

Subset Sum Pseudo-code

- ◆ $S \leftarrow$ Boolean two-dim array of size $n \times (\sum a_i + 1)$
- ◆ $S[j, k]$ denotes whether there is a subset of j nos. with sum $= k$
- ◆ Say, range of j , $1 \leq j \leq n$
- ◆ Range of k , $\text{Neg} \leq k \leq \text{Pos}$ // Neg - sum of -ve number & Pos - sum of +ve numbers.

Subset Sum Pseudo-code

- ◆ Initialization:

- ◆ $S[1,k] \leftarrow \{\text{true for } k = a_1; \text{ false for all other values of } k\}$

for($j = 2$ to n)

for($\text{Neg} \leq k \leq \text{Pos}$)

$S[j, k] = S[j - 1, k] \text{ or } S[j - 1, k - a_j]$

Subset Sum Pseudo-code

To construct the desired set

Assume, $S[n,0]$ is true

$Sum \leftarrow 0$

for($j = n$ to 1)

if $S[j - 1, Sum] = \text{true}$; don't take a_j

Else if $S[j - 1, Sum - a_j] = \text{true}$; take a_j and $Sum \leftarrow Sum - a_j$

Knapsack Problem

- ◆ We are given a knapsack of weight W (a.k.a capacity), and n objects of weight $\{w_1, w_2, \dots, w_n\}$ and values $\{v_1, v_2, \dots, v_n\}$
- ◆ Select a subset S s.t.

$$\text{◆ } \sum_{i \in S} w_i \leq W$$

Total value $\sum_{i \in S} v_i$ is maximized.

Knapsack - Ex.

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7
W = 11		

Optimal - {3,4},
with Value - 40

Knapsack - Greedy

- ◆ Pick repeatedly the item with largest $\frac{v_i}{w_i}$ ratio!

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7
W = 11		

Greedy sol. $\{5,3,1\} = 35$

Opt. ≈ 40

Optimal substructure - Attempt - 1

- ♦ $\text{OPT}[i]$ = Optimal solution considering the items $\{1, 2, \dots, i\}$ and knapsack of size W .
- ♦ Case 1: Item i not part of the $\text{OPT}[i]$.
 - ♦ $\text{OPT}[i] = \text{OPT}[i - 1]$
- ♦ Case 2: Item i is part of the sol. $\text{OPT}[i]$.
 - ♦ Inclusion of i doesn't mean that we need to reject $i - 1$
 - ♦ We can't say also - $\text{OPT}[i] = \text{OPT}[i - 1] + v_i$ Why?

Optimal substructure - Attempt - 1

- ♦ $\text{OPT}[i]$ = Optimal solution considering the items $\{1, 2, \dots, i\}$ and knapsack of size W .

- ♦ Case 1: Item i not part of the $\text{OPT}[i]$.

If we select item i , we only have $W - w_i$ space left.

- ♦ $\text{OPT}[i] = \text{OPT}[i - 1]$

- ♦ Case 2: Item i is part of the sol. $\text{OPT}[i]$.

- ♦ Inclusion of i doesn't mean that we need to reject $i - 1$

- ♦ We can't say also - $\text{OPT}[i] = \text{OPT}[i - 1] + v_i$ Why?

Optimal Substructure - Attempt - 2

- ◆ Let's add another "dimension" to the subproblem definition.
- ◆ $\text{OPT}[i, w]$: Optimal solution considering the items $\{1, 2, 3, \dots, i\}$ and residual capacity w , $[0 \leq w \leq W]$.

		0	1	2	3
i	1				
	2				
	3				

This entry indicates the max.
Profit obtained from items $\{1, 2\}$
With capacity = 2

Optimal Substructure - Attempt - 2

- ♦ $\text{OPT}[i, w]$: Optimal solution considering the items $\{1, 2, 3, \dots, i\}$ and residual capacity w , $[0 \leq w \leq W]$.
- ♦ For $i = 1, \dots, n$ & $w = 0, 1, 2, \dots, W$
- ♦ Case 1: item i is not part of $\text{OPT}[i, w]$; $\text{OPT}[i, w] = \text{OPT}[i-1, w]$
- ♦ Case 2: item i is part of $\text{OPT}[i, w]$; $\text{OPT}[i, w] = \text{OPT}[i-1, w - w_i] + v_i$

Dynamic Programming

Knap (n, W)

for w = 0 to W // base case

$M[0, w] = 0$

for i = 1 to n // outer loop for item index

for w = 0 to W // inner loop for residual capacity

if (w > w)

$M[i, w] = M[i - 1, w]$

else

$M[i, w] = \max\{M[i - 1, w], M[i - 1, w - w] + v\}$

Return M[n, W]

Running time

- ◆ Knapsack is NP-hard.
- ◆ $O(n.W)$
- ◆ Pseudopolynomial time.
- ◆ PTAS exists.